

UNIT 4-Lecture 8 : system Validation

Introduction to System Validation

Definition

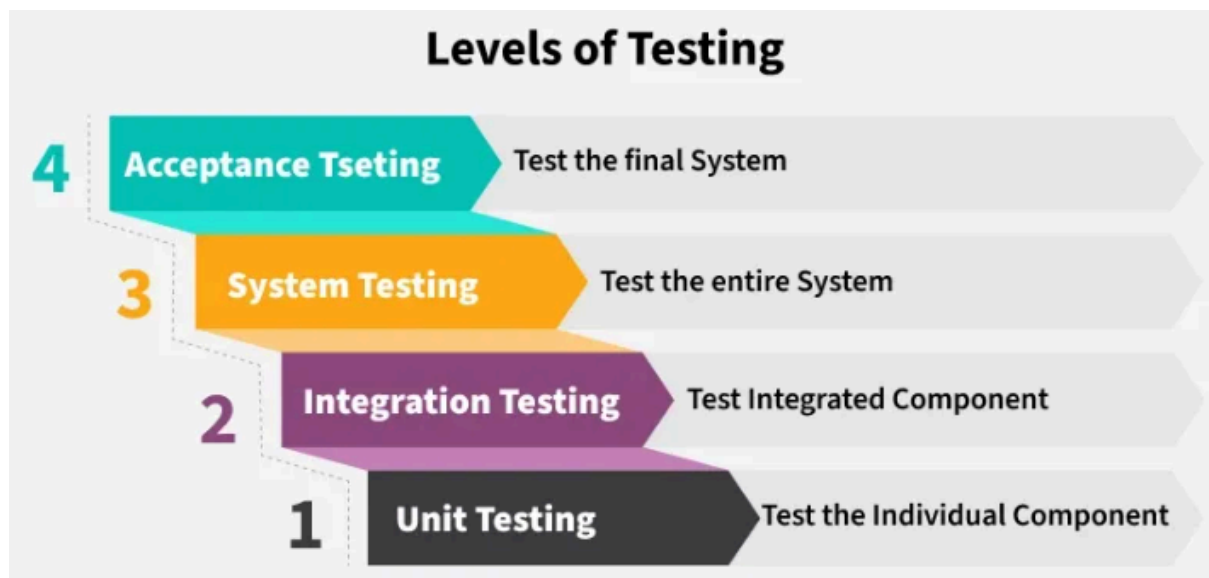
System Validation is the process of evaluating a completely integrated software system (or software + hardware system) to determine whether it satisfies the specified business requirements, user needs, and intended use in its real operational environment.

Key Question Answered by Validation

“Are we building the right product?”

(Does the final system actually solve the customer’s problem and meet their expectations?)

Types Of Testing:



1. Unit Testing

Focus: Test for coding/construction errors prior to quality engineering

Goal: Verify that the smallest individually executable code units (functions, methods, classes, or modules) work correctly in isolation.

- **Who performs it?** Usually done by the developers themselves (white-box testing).
- **When?** During the coding phase, often written before or alongside the code (TDD – Test-Driven Development).

- **What is tested?**
 - Algorithm and logic correctness
 - Data structures
 - Interfaces (parameters, return values)
 - Independent paths / branches (decision coverage)
 - Boundary conditions (edge cases, nulls, empty inputs)
 - Error handling (exceptions, invalid inputs)
- **Tools commonly used:** JUnit (Java), NUnit (.NET), pytest (Python), Google Test (C++), Mocha/Jest (JavaScript), etc.
- **Key principle:** In Object-Oriented environments, tests are written at the class level using constructors and destructors.
- **Advantages:** Finds bugs very early, makes refactoring safe, serves as living documentation.

2. Integration Testing

Focus: Verify the interface and interaction between components against the software design.

- **Who performs it?** Performed by programmers or dedicated integration/test engineers.
- **Goal:** Catch interface mismatch issues, data flow problems, and integration defects that unit testing cannot catch.
- **Common problems found:**
 1. Incompatible interfaces
 2. Incorrect parameter passing
 3. Wrong assumptions about another module's behavior
 4. Database/connection issues
- **Approaches / Strategies:**
 1. **Big Bang Approach** – All units are integrated at once and tested. – Fast but chaotic; hard to locate defects.
 2. **Incremental / Iterative Approach** (preferred)
 - **Top-down:** Start from top-level modules, use stubs for lower ones.
 - **Bottom-up:** Start from lowest modules, use drivers for higher ones.
 - **Sandwich / Hybrid:** Combination of top-down and bottom-up.
- **Goal of iterative approach:** Software components are integrated iteratively until the complete system is built (top-down or bottom-up).

3. System Testing

Focus: Test a completely integrated system to verify compliance with Software Requirement Specifications (SRS).

- **Who performs it?** Independent testing team (black-box + sometimes grey-box).
- **Scope:** Entire system as a whole (in-test + end-to-end).
- **Types of System Testing:**
 1. **Smoke and Sanity Testing**
 - Quick check to confirm the build is stable enough for further testing.
 - Ensures major functionalities work and the most important functions are operational.
 2. **Software Performance Testing**
 - Load, Stress, Scalability, Stability, Reliability testing.
 - Measures response time, throughput, resource utilization under various loads.
 3. **Functional and Non-functional Testing**
 - Functional: Does it do what it is supposed to do? (covers all scenarios including failure paths)
 - Non-functional: Security, Usability, Compatibility, Maintainability, etc.
 4. **Recovery Testing**
 - Force system crash or shutdown (planned/unplanned) and verify it recovers gracefully.
 5. **Destructive Testing**
 - Application is made to fail deliberately to test robustness.
 - Done in a controlled manner.
 6. **Installation Testing**
 - Verify software installs correctly with all components.
 - Verify the system still works even if some non-critical components are removed.
 - Check uninstallation/cleanup.
 7. **Usability Testing**
 - Ease of use from the end-user's perspective.
 - How much effort/time is required to learn and operate the system.
 8. **Cloud Testing**
 - Performed on cloud infrastructure.
 - Question to consider: Could performing cloud testing skip any of the above traditional tests? (Answer: No – cloud is just the environment; core testing types remain necessary.)
- **Typical Process in System Testing:**
 1. Test Environment Setup
 2. Create Test Cases & Test Data
 3. Execute Test Cases
 4. Defect Reporting and Logging
 5. Defect Fix → Regression → Retest

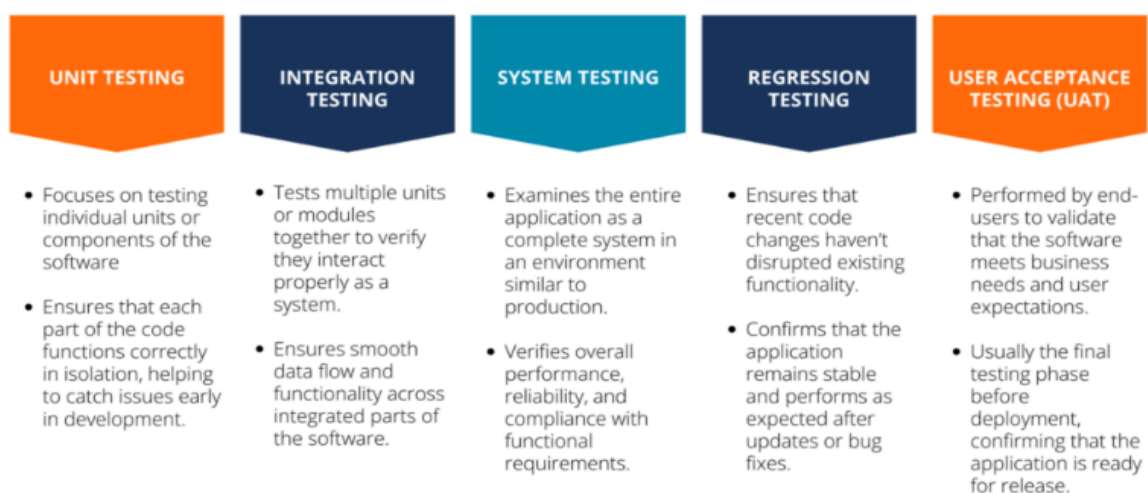
4. Acceptance Testing

Focus: Done by the system providers/users/customers to determine if the system meets their needs and is ready for deployment.

- **Types:**
 - **User Acceptance Testing (UAT)** – Real users in real environment.
 - **Operational Acceptance Testing (OAT)** – Admin, backup, recovery, etc.
 - **Alpha Testing** – Done at developer's site by internal users/testers.
 - **Beta Testing** – Done at customer's site by real end-users (pre-release versions).
 - **Contract Acceptance Testing** – Against contract terms.
 - **Regulation/Compliance Acceptance Testing** – Legal or industry standards.
- **Goal:** Gain confidence that the system is ready for production and the customer will accept it.

Level	Who Does It	Focus	Knowledge of Code	Typical Techniques
Unit Testing	Developers	Individual modules/functions	White-box	JUnit, pytest, code coverage
Integration Testing	Developers / Testers	Interfaces between modules	White/Grey-box	Stubs, Drivers, Big-bang/Incremental
System Testing	Independent Testers	Entire system vs requirements	Mostly Black-box	Functional, Performance, Recovery, etc.
Acceptance Testing	Users / Customers	Business needs & readiness	Black-box	UAT, Alpha/Beta testing

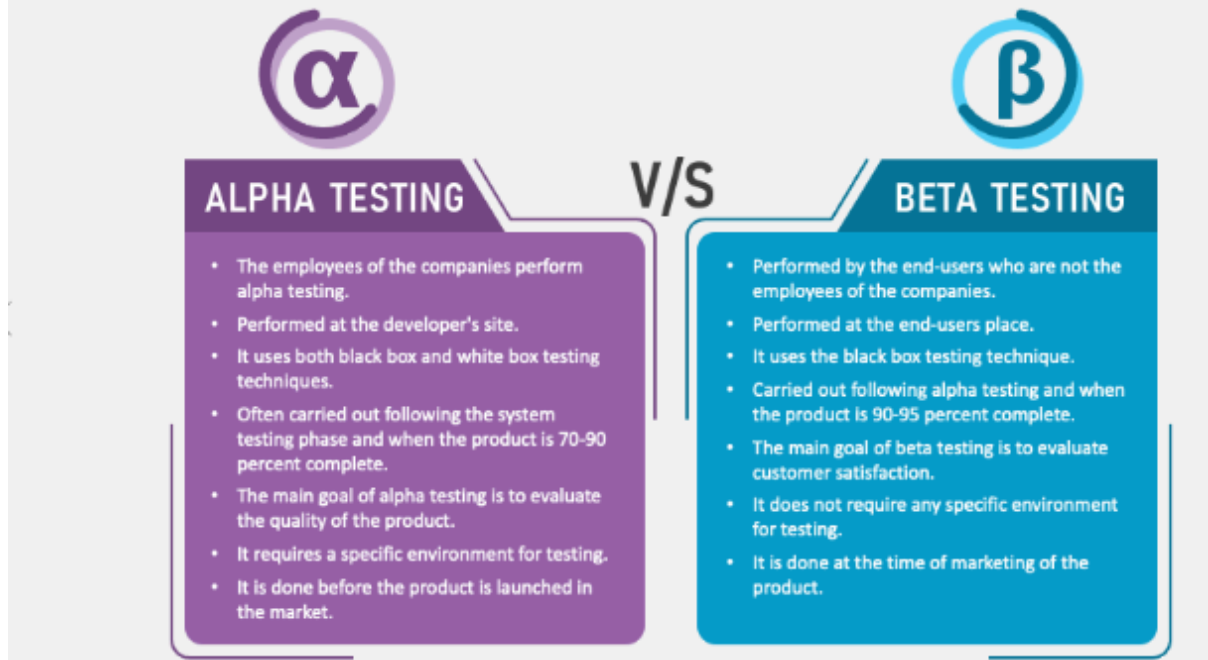
FUNCTIONAL TESTING TYPES



System Testing – Types (Continued)

Type	Detailed Explanation	Key Objectives & Examples
Localization Testing	Verifies that the software correctly adapts to a specific language, region, and culture (locale).	→ Date/time formats, currency, number formatting → Text direction (LTR vs RTL) → Translation quality & cultural appropriateness → Local laws (e.g., GDPR in EU)
Compliance Testing	Ensures the product meets internal organizational standards and external regulatory/industry standards.	→ ISO 9001, ISO 27001, HIPAA, PCI-DSS, FDA 21 CFR Part 11, SOC 2, etc. → Audit trails, data retention policies
Scalability Testing	Determines the system's ability to scale up (handle more users/load) or scale down gracefully.	→ Vertical scaling (add CPU/RAM) → Horizontal scaling (add servers) → Auto-scaling behavior in cloud
Boundary Testing	Tests values at the extreme edges of input domains (boundary value analysis).	→ Min, min-1, min+1, max-1, max, max+1 → 0, 1, 99, 100 for a field that accepts 1–100
Regression Testing	Re-execution of previously passed test cases after any change to ensure no new defects have been introduced.	→ Automated regression suite is ideal → Run after every bug fix, code refactor, or new feature
Startup & Shutdown Testing	Verifies graceful start and stop of the system, especially when unexpected interruptions occur.	→ Does the system recover cleanly after power loss? → Are files/tables left in consistent state?
Platform Testing	Tests the application on different operating systems, browsers, devices, and versions.	→ Windows 10/11, macOS, Linux → Chrome, Firefox, Edge, Safari → Mobile: iOS 16/17/18, Android 12/13/14
Load Testing	Applies expected real-life load to measure performance (response time, throughput).	→ Simulate 5,000 concurrent users for 2 hours
Stress Testing	Pushes the system beyond normal/maximum limits to find the breaking point.	→ Gradually increase load until CPU > 99%, memory exhausted, or crashes occur
Security Testing	Identifies vulnerabilities and ensures data and resources are protected.	→ OWASP Top 10 (SQLi, XSS, CSRF, etc.) → Authentication, authorization, encryption, penetration testing

ALPHA TESTING VS BETA TESTING



2. Acceptance Testing

Definition

Final level of testing performed on the complete, integrated system to validate that it is ready for delivery and use.

Core Purpose

- Provide confidence that the delivered system satisfies business requirements
- Final quality gateway before go-live
- Each test case represents a real business scenario or user journey

Key Characteristics

- Tests are created through collaboration between customers, business analysts, testers, and developers
- Executed in a production-like environment
- Verifies the completeness of user stories during a sprint (in agile)
- Focus is on "Does this solve my business problem?" not just technical correctness

Critical Question

What happens if undetected errors are found after acceptance?

→ Very expensive fixes, damaged customer trust, possible contract penalties, emergency hotfixes in production → Therefore, acceptance is the last safety net.

3. Acceptance Testing Categorization

Category	Who Performs It	Environment	Key Focus
Acceptance / Qualification Testing	Customer / End users	Production-like	Does the system meet business needs and user expectations?
Installation Testing	Customer IT team or vendor	Target production environment	Clean installation, upgrades, uninstallation, config files, permissions
Alpha & Beta Testing	Alpha: Internal users at developer site Beta: Real external users	Alpha: Controlled lab Beta: Real customer sites	Real-world usage, usability feedback, missing requirements, bugs not found in lab
Performance Testing (in Acceptance context)	Customer performance team	Production-like load & data	Confirm SLA response times, throughput, resource usage under realistic conditions

4. Integration Testing – Why It Is Needed

Simple Definition of Integration

Combining 2 or more software units and verifying they work together as expected.

Why plain “integration” is not enough – we need dedicated integration testing

Even if every unit works perfectly in isolation, problems appear only when units interact.

Common Integration Problems (not visible in unit testing)

- New modules interact in ways developers never anticipated
- Modules were developed by different teams → different assumptions
- Data format mismatches (e.g., date as String vs Date object)
- Incorrect error handling across module boundaries
- Performance bottlenecks only appear when modules call each other
- Database deadlocks, transaction issues
- API version mismatches

Classic Integration Risks

- “It works on my machine” syndrome
- Big-bang integration → everything breaks at once → impossible to debug
- Cascade of interdependencies → one small change breaks distant modules

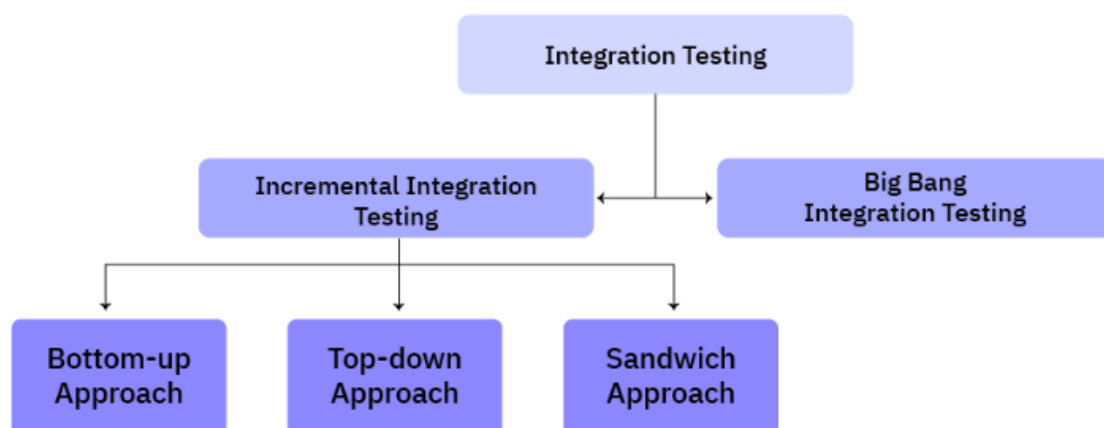
Consequences of Poor Integration Testing

- Defects found very late → extremely expensive to fix
- Delayed releases, missed deadlines
- Customer loses confidence

Best Practices to Reduce Integration Risk

- Continuous Integration (CI) + automated integration tests
- Use stubs/drivers only when necessary
- Test critical and high-risk interfaces first
- Maintain interface contracts (OpenAPI, JSON schema, etc.)

Integration Testing



1. Why We Cannot Just “Start with Plain Integration”

Plain / Naive Integration =

Just combine 2 or more units and pray they work together.

Reality – Why this fails almost every time:

- New problems appear only when modules are wired together for the first time
- Many modules are developed by different teams → they have never “talked” to each other before
- If everything is integrated at once (“Big Bang”), all problems explode simultaneously

- This creates a cascade of interdependencies → one bug triggers dozens of failures
- Result → Impossible to isolate root cause, debugging becomes a nightmare
- Cannot find and solve problems one-at-a-time

Conclusion: Pure “plug-and-play” integration without proper testing is one of the biggest reasons for project delays and failures.

2. What Do You Think of “Phased” Integration? (The Sarcastic Slide)

“Big-Bang” Integration (the joke approach)

- Design, code, test, debug each class/unit/subsystem completely separately
- Keep them isolated until the very end
- Finally combine everything in one go
- “Hope for the best” (picture of supernova explosion → exactly what happens)

Why Big-Bang is a disaster in real projects > 5–10 developers:

Issue	Consequence
All defects surface at the same time	Impossible to localize which module caused which failure
Fault localization time explodes	Teams blame each other → politics, finger-pointing
Fixing one bug breaks ten others	Never-ending regression loop
Schedule slips by weeks/months	Missed deadlines, huge overtime, burnt-out teams
Management panic → more pressure	Quality drops even further

Real-world statistic: Projects that use pure Big-Bang integration are 5–10× more likely to miss deadlines.

3. Formal Definition of Integration Testing

Integration Testing =

Verifying software quality by testing two or more dependent software modules as a combined group.

Key points:

- Can be quite challenging because combined units can fail in more places and in far more complicated ways than individual units

- You may need to create stubs (fake lower-level modules) or drivers (fake higher-level callers) to simulate behavior of pieces that don't exist yet or are unstable

4. Incremental Integration – The Professional & Recommended Way

Core idea:

Design → Code → Test → Debug a new component → Integrate it with the existing (already tested) part of the system → Test the new combined behavior → Repeat

Common incremental strategies:

1. Top-Down (start from UI/main module, use stubs for lower layers)
2. Bottom-Up (start from lowest utilities/DB layers, use drivers for UI)
3. Sandwich / Hybrid (both directions meet in the middle)
4. Feature-by-Feature (agile/DevOps style)
5. Backbone / Critical Path first (most important flows first)

Process loop:

- Can start with a very thin “functional skeleton” or “walking skeleton”
- System grows feature by feature or layer by layer
- And we incrementally “flesh it out”

5. Benefits of Incremental Integration (Why Teams Love It)

Benefit	Explanation
Errors are easier to isolate, find & fix	Only one new module is added at a time → failure clearly points to the newcomer or its interface
Reduces developer bug-fixing load	Defects are found and fixed early, not piled up at the end
System is always in a (relatively) working state	Stakeholders can see progress continuously → higher confidence
Good for customer & developer morale	Visible progress, fewer last-minute crises
Enables Continuous Integration / CI-CD	Automated builds and tests run after every small integration
May need temporary “stub” versions	Acceptable trade-off — stubs are replaced later when real components are ready

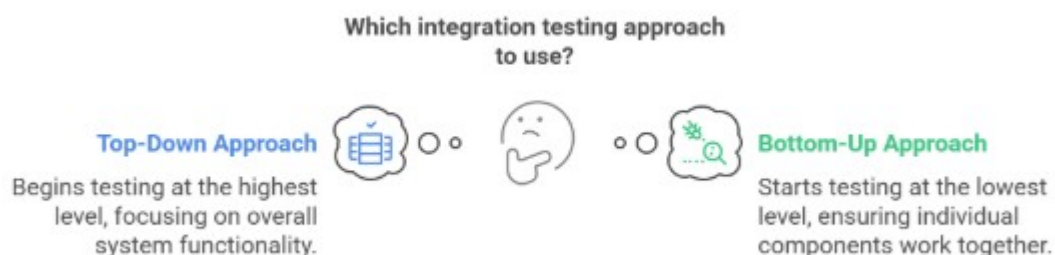
6. Quick Comparison Table

Aspect	Big-Bang Integration	Incremental Integration
When integration happens	Only once, at the very end	Continuously, piece by piece
Fault localization	Extremely difficult	Very easy (only new piece can be guilty)
Debugging time	Huge	Minimal
Risk level	Very High	Low to Moderate
Suitable project size	Tiny (< 3 developers)	Small → Very Large, Enterprise
Parallel development support	Poor	Excellent
Need for stubs/drivers	Rare	Frequent but temporary
Team stress	Sky-high at the end	Evenly distributed
Delivery confidence	Low until the final explosion	High from early stages

Best Practices for Successful Integration Testing (Real-World Tips)

- Maintain clear interface contracts (API specs, Swagger/OpenAPI, database schemas)
- Automate integration test suite and run it on every commit (CI pipeline)
- Prioritize high-risk and critical modules for early integration
- Use feature toggles to integrate unfinished features safely
- Keep stubs as thin and realistic as possible
- Version control everything — including test data and environment configs

1. Incremental Integration Strategies – The Three Main Approaches



How It Works	Visual Flow	Advantages	Disadvantages / Challenges
Start from the top (UI / main control modules) and integrate	Start at top → work down	• Early working prototype of UI/flow • Business	• Needs many stubs for lower layers • Stubs can hide real bugs

downwards		stakeholders see progress quickly • High-level design flaws found early	• Core logic tested very late
Start from the bottom (DB, utilities, services) integrate upwards	Start at bottom → work up	• Real core components tested early • Fewer/no stubs needed • Drivers are usually simpler than stubs	• Needs drivers to simulate UI/calls • No working UI until very late • Major design flaws in flow discovered late
Combine both: integrate top-down and bottom-up simultaneously; they meet in the middle	Two fronts moving toward center	• Most practical in real projects • Balances early UI demo + solid foundation • Most common in modern & agile teams	• Requires careful planning & coordination • Slightly more complex test environment setup

2. Regression Testing



Definition

Testing executed after any change (bug fix, new feature, refactoring, config change) to confirm that the existing, previously working functionality is still working correctly.

Golden Statement

“Regression testing is testing the system to check that changes have not ‘broken’ previously working code.”

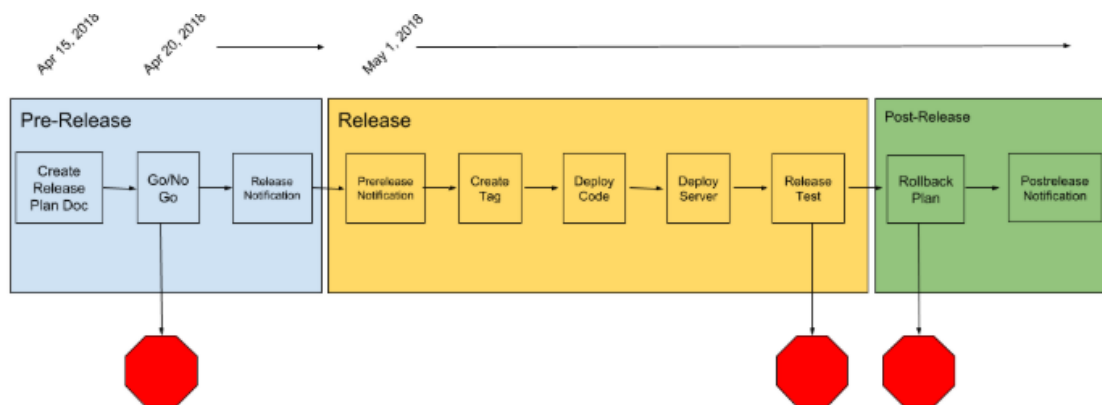
Key Characteristics:

Aspect	Explanation
Manual vs Automated	Manual regression is extremely expensive and slow Automated regression is fast, repeatable, and cheap after initial investment
Scope	Can be full regression (entire test suite) or selective (risk-based)
When it runs	After every code change, every build, every pull request, nightly, before release
Best Practice	All automated regression tests must pass ("green") before the change is allowed into main branch

Real-world Rule (widely used in mature teams)

“No code gets merged until the full regression suite passes successfully.”

3. Release Testing vs System Testing



Aspect	System Testing	Release Testing
Who performs	Internal QA / Test team that has been involved throughout	Separate, independent team that has NOT been involved in development
Primary goal	Find as many bugs as possible	Check that the system is good enough for external release (validation-focused)
Environment	Test/staging environments	Production-like or actual pre-production environment
Mindset	“Break the system” – exploratory, deep, negative testing	“Confirm it is safe to ship” – disciplined, documented, sign-off
Test cases	Thousands of detailed technical cases	Focused set of business-critical, end-to-end scenarios
Outcome	Bug reports → fix → retest	Go / No-Go decision for release

4. Features Tested by Scenario – Real-World Example (Healthcare System)

These are typical end-to-end business scenarios used in System / Acceptance / Release testing:

Scenario	Why It Is Critical
Authentication by logging on to the system	If login fails → entire system inaccessible
Downloading and uploading of specified patient records to a laptop	Doctors/nurses work offline in remote areas
Home visit scheduling	Core business workflow
Encryption and decryption of patient records on a mobile device	Legal requirement (HIPAA, GDPR, etc.)
Record retrieval and modification	Daily clinical work
Links with the drugs database that maintains side-effect information	Patient safety – wrong drug or allergy can be fatal
The system for call prompting (reminders & alerts)	Ensures patients get timely care and medication

These scenarios are usually turned into Acceptance Test Cases and are executed with real or realistic data in a production-like environment.

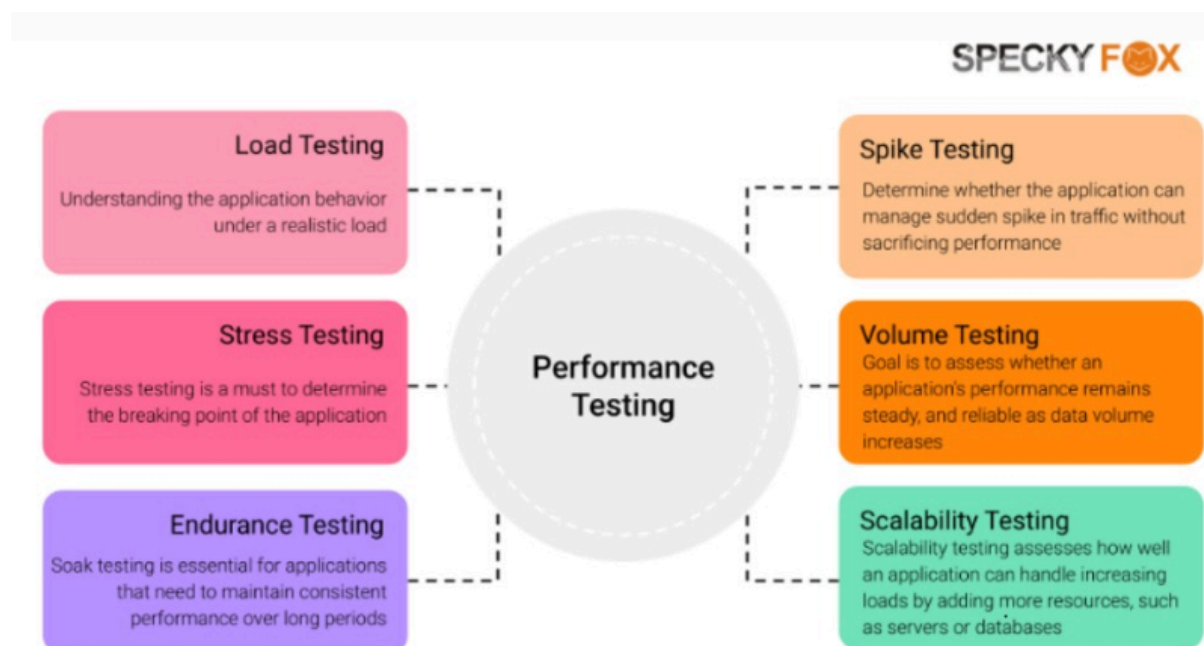
1. Performance Testing



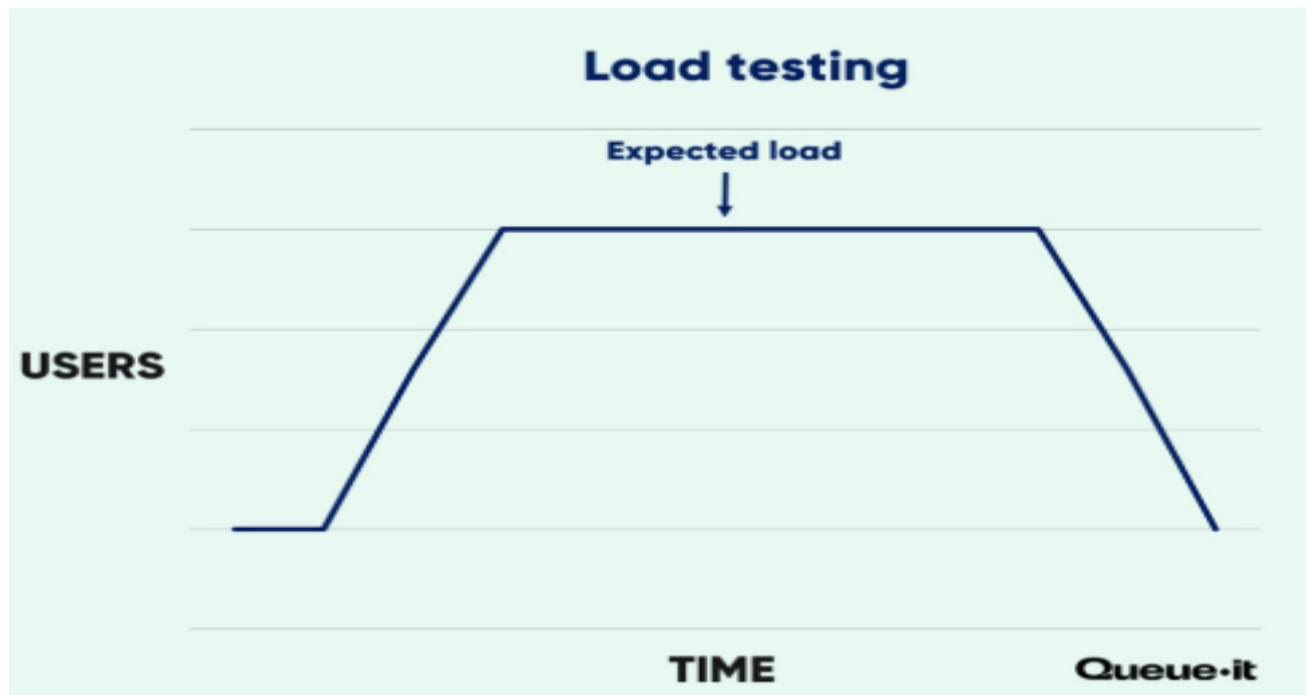
Performance testing is a *non-functional* testing technique that exercises a system and then, measures, validates and verifies the response time, stability, scalability, speed, and reliability of the system in production-like environment. It is a process in which we try to find the bottleneck for our application and its certain features, i.e. at what particular stress point our web application will crash.

Key notes:	Detailed Explanation
Part of release testing – focuses on emergent properties (performance, reliability, scalability)	These qualities cannot be tested at unit level – they only appear when the full system is running under real/realistic load.
Tests should reflect the profile of use of the system	Use actual business transactions and real user behavior patterns (e.g., 60% read, 30% search, 10% write), not artificial traffic.
Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable	This is called Load → Volume → Stress → Soak progression: • Load Testing → normal/peak expected load • Volume Testing → huge data sets • Stress Testing → beyond limits to find breaking point • Soak/Endurance → long duration at high load to detect memory leaks, degradation
Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behavior	Goal: Observe graceful degradation, recovery mechanism, error messages, auto-scaling triggers, and exact breaking point (e.g., 12 000 concurrent users → 503 errors start).

Most Common Performance Test Types (Quick Table)

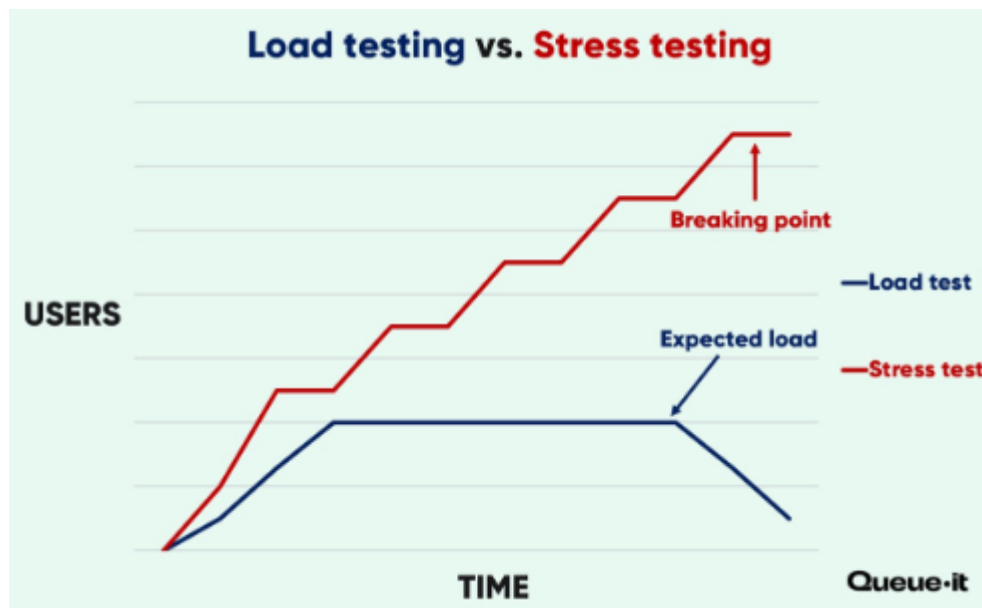


1. Load Testing



Key points:	“Performance tests usually involve planning a series of tests where the load is steadily increased...”
Definition	Testing the system under anticipated normal and peak load conditions that are expected in production.
Goal	Verify that the system meets the defined performance criteria (response time, throughput, resource utilization) when subjected to the expected real-life load.
Typical Load Levels	• Average daily load • Peak business hours load (e.g., 5 000 concurrent users) • Seasonal peaks (Black Friday, salary day, etc.)
Key Metrics	• Average response time ≤ 2 sec • 95th percentile ≤ 4 sec • Throughput (transactions/sec) • CPU / Memory / DB connections $< 80\%$
Duration	Usually 1–4 hours (long enough to reach steady state)
Example	An e-commerce site must handle 10 000 concurrent users browsing and 800 users checking out simultaneously without response time > 3 seconds.

2. Stress Testing



Key points:	"Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behaviour."
Definition	Intentionally pushing the system beyond its defined limits to determine the breaking point and observe failure behaviour.
Goal	• Find the maximum capacity • Observe how the system fails (graceful vs crash) • Verify auto-recovery, failover, queue mechanisms • Identify the weakest component (app server, DB, cache, network)
Load Pattern	Gradually or suddenly increase load until errors appear (e.g., 503, timeouts, crashes).
Key Observations	• At what exact load do errors start? • Does it recover when load is reduced? • Are error messages user-friendly?
Duration	Until system breaks or stabilises at overload (usually 30 min – 2 hours).
Example	Banking app is stress-tested from 5 000 → 20 000 → 50 000 users until the database connections are exhausted and the app starts queueing requests.

3. Spike Testing



What is it?

Spike testing means suddenly throwing a **huge number of users** at the system for a very short time and then taking them away – just like real life.

Real-life examples (everyone understands these):

- 12:00 noon → Flipkart/Amazon flash sale starts → 5 lakh people click “Buy” together
- IPL ticket sale opens → 3 lakh people attack the website in 10 seconds
- A big celebrity posts on Instagram → millions visit the linked page instantly

What we want to check:

1. Does the website/app crash immediately? (No!)
2. Does it show a proper “Please wait” or queue page instead of error?
3. Do extra servers start automatically within 1–2 minutes?
4. When the rush ends, does everything become fast again?

How we do the test (simple steps):

- Normal day → 1 000 users
- Suddenly → 1 lakh users in 10 seconds
- Keep 1 lakh users for 5–10 minutes
- Suddenly remove them → back to 1 000 users

What we watch:

- Did any user get “503 error” or blank page?
- How many seconds did it take to become normal again after the rush?

4. Scalability Testing

Scalability testing is a type of load testing that involves measuring the ability of an application to scale up or down in reaction to load.

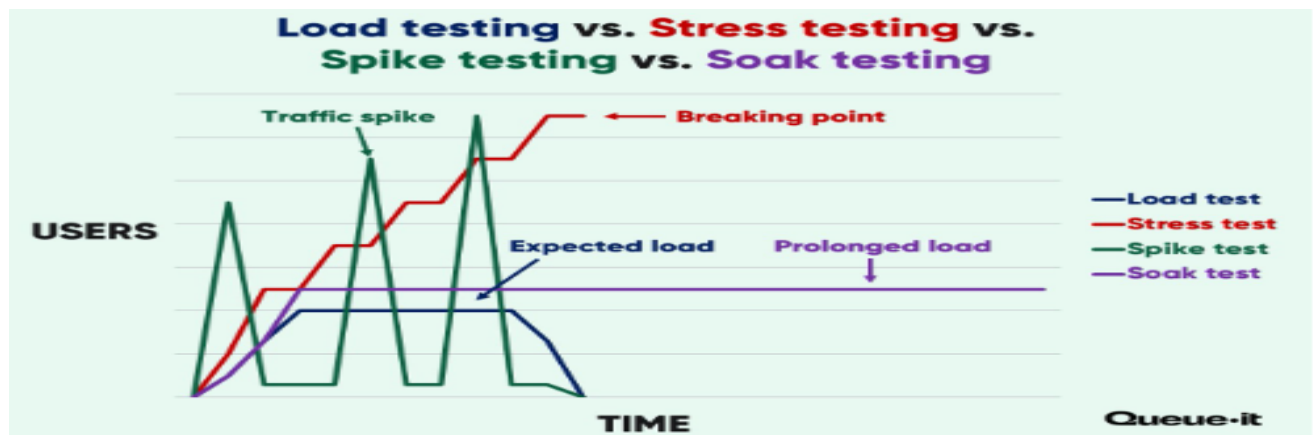
The objective of scalability testing is to ensure the system can handle an expected increase in users, data volume, or transactions, and to determine the point at which the application cannot continue to scale.

Like spike testing, scalability testing involves applying variable levels of load. But unlike spike testing, scalability tests give the system time to scale up in response to load. Another key difference is that scalability testing can be performed on software, hardware, and the database.

Scalability testing can also be used to determine the effectiveness of server scaling and autoscaling.

Key points:	Implied under performance testing series (steady increase until unacceptable)
Definition	Determining how well the system scales up or out when resources or users are increased.
Two Types	• Vertical Scalability – Adding more CPU/RAM to existing servers • Horizontal Scalability – Adding more server nodes
Goal	• Prove that performance remains acceptable when user load doubles, triples, etc. • Validate auto-scaling rules in cloud
Load Pattern	Step-wise increase (5k → 10k → 20k → 40k users) with 15–30 min plateau at each step.
Key Metrics	• Response time at 2×, 5×, 10× load • Cost per user • Time taken for new instances to join cluster
Example	Netflix tests whether adding 10 more EC2 instances brings 99th percentile latency back under 200 ms when users jump from 50k to 200k.

5. Soak (Endurance / Reliability) Testing



Slide Reference	Part of the “series of tests” where system is run for extended period
Definition	Running the system at high (or slightly above) expected peak load for a prolonged period (24–72 hours or even 7 days).
Goal	Detect problems that only appear after long running time: • Memory leaks • Database connection leaks • Log file / temp file growth • Thread pool exhaustion • Gradual performance degradation
Load Pattern	Constant high load (e.g., 80–90% of peak) for many hours/days.
Key Metrics	• Response time trend over time • Memory/CPU trend (should be flat) • Garbage collection behaviour • Number of errors after 24/48/72 hrs
Example	A trading platform is run at 8 000 concurrent users for 72 hours over a weekend to ensure there is no heap memory growth or connection pool leak.

Quick summary Table:

Type	Objective	Typical Duration	Key Metrics Measured
Load Testing	Does it work under expected load?	1–4 hours	Response time, throughput, CPU/memory
Stress Testing	What is the breaking point?	Until failure	Max users, error rate, crash point
Spike Testing	Sudden huge load (e.g., flash sale)	Seconds–minutes	Recovery time after spike
Scalability Testing	Can it handle 2×, 10× growth?	Hours	Auto-scaling behavior
Soak/Endurance	Does performance degrade over days? (memory leaks, DB bloat)	24–72 hours	Resource usage trend over time

Penetration Testing

What is Penetration Testing?

Penetration testing (also called pen-testing or ethical hacking) is a **legal, planned attack** on your own system to find security holes before bad hackers do.

It is like hiring a professional thief to try to break into your house at night so you can fix the weak doors and windows before a real thief comes.

Goal: Discover real weaknesses in the application, server, network, or people and prove that they can actually be exploited, and give proof with screenshots or videos.

It is different from normal security scanning because a human expert thinks and acts like a real attacker – combining multiple small issues into one big successful attack.

Three Types of Penetration Testing

1. **White-box testing** The tester is given everything – source code, architecture diagrams, admin passwords, network maps. It is like attacking from the inside. Used when you want the deepest and most complete check (common in banks, medical software, hardware products).
2. **Black-box testing** The tester knows nothing – only the website URL or IP address, exactly like a real hacker from the internet. Most realistic for external attacks.
3. **Gray-box testing** The tester gets limited information – for example, a normal user login or some API documentation. This is the most common and practical type because real attackers often start with a stolen username/password.

Different companies choose different types depending on what they want to test.

How a Real Pen-Test is Done – Step by Step

Reconnaissance – Collect information Google search, LinkedIn, Shodan, company website, WHOIS – anything publicly available.

1. **Scanning** – Probe the target Use tools like Nmap to find open ports, running services, software versions.
2. **Vulnerability Assessment** – Find weak spots Run scanners to list possible bugs (out-of-date software, weak passwords, misconfiguration).
3. **Exploitation** – Actually break in Try real attacks: SQL injection, login bypass, file upload, privilege escalation, etc. Take screenshots or record video as proof.
4. **Post-exploitation** – Move deeper Once inside, try to get admin rights, steal data, or control more servers.

5. **Reporting** – Give a proper report List every issue with risk level (Critical/High/Medium/Low), proof (screenshot), and exact steps to fix it.

Why Companies Pay for Pen-Testing

- Finds dangerous bugs that normal testing or scanners miss
- Required by law or customers (PCI-DSS for cards, HIPAA for hospitals, etc.)
- Gives proof to management and customers that security is taken seriously
- Helps the company fix the most dangerous things first (Red Team style)
- In many real breaches, a pen-test 6–12 months earlier would have stopped the attack
- Cheaper than paying fines or losing customers after a real hack

Popular Free Tools Used by Pen-Testers Nmap → finds which doors (ports) are open on servers

- Metasploit → huge library of ready attacks (Community version is free)
- OWASP ZAP → free tool to attack web applications automatically + manually
- SQLMap → automatically finds and exploits SQL injection bugs
- Burp Suite (Community) → most loved tool to watch and change web traffic
- Wireshark → sees every packet going in and out
- John the Ripper / Hashcat → cracks passwords

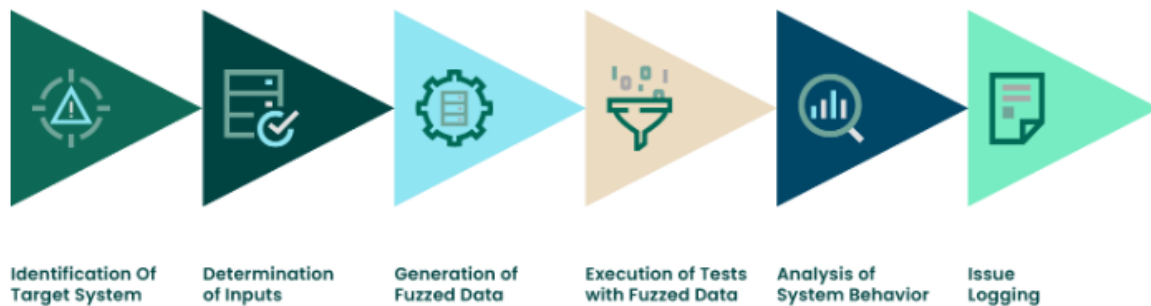
Real-Life Examples

2023 Global Enterprise → 372 super dangerous holes found in 52,000 servers.
Pen-test showed attackers could have taken everything.

- Marriott Hotel 2020 → Hackers stole 5.2 million guest records because of one old weak system. A pen-test would have found it years earlier.
- IoT devices (smart cameras, bulbs, cars) → Pen-testers regularly show they can switch on your camera or open your car door remotely because of bad code from third-party suppliers.

Fuzzing

How Does Fuzz Testing Work?



What is Fuzzing?

Fuzzing (or fuzz testing) is an **automated way of finding bugs by throwing thousands or millions of random, wrong, broken, or unexpected inputs** at a program and watching if it crashes, hangs, or behaves strangely.

It is like a monkey banging randomly on a keyboard – most of the time nothing happens, but sometimes the whole application crashes or a serious security hole is found.

Purpose: Discover **hidden crashes, memory corruption, buffer overflows, assertion failures, and security bugs** that normal test cases never reach.

Types of Fuzzers (Slide: Types)

1. **Mutation-based (Dumb fuzzing)** Take normal valid files/inputs and randomly flip bits, change lengths, insert garbage. Very simple but surprisingly effective.
2. **Generation-based (Smart fuzzing)** The fuzzer understands the format (PDF, JPEG, HTTP, etc.) and creates brand-new inputs from scratch following the rules – but also breaks the rules on purpose.
3. **White-box fuzzing** The fuzzer can see the source code and tries to reach every line and branch.
4. **Gray-box fuzzing** Most popular today – the fuzzer only sees which parts of the code were executed (coverage) but not the full source.
5. **Black-box fuzzing** No knowledge at all – just keep sending random data to the program or network port.

How Fuzzing Actually Works – Step by Step

Input Generation → Create or mutate thousands of test files/requests per second.

1. **Program Execution** → Run the target program with each weird input.
2. **Monitoring/Crash Detection** → Watch for crashes, memory errors, timeouts, or assertion failures.

3. **Feedback Loop (Coverage-guided)** → The fuzzer remembers which inputs discovered new code paths and creates more similar inputs to dig deeper. This loop runs 24×7 for days or weeks → keeps finding deeper and deeper bugs.

Why Fuzzing is So Powerful

Finds **deep, rare bugs** that humans never think of (Google, Microsoft, Mozilla find hundreds of critical bugs every year using fuzzing).

- Works on **everything** – browsers, SSL libraries, image parsers, kernel drivers, network protocols.
- Runs non-stop – perfect for CI/CD pipelines.
- Discovered bugs in supposedly “super secure” software that existed for 10–15 years.
- Much cheaper and faster than manual security audits for low-level code.

Popular Free Fuzzing Tools

(used by Google, Microsoft, Mozilla).

- **libFuzzer** → Built into LLVM/Clang, very fast for C/C++ programs.
- **Honggfuzz** → Another powerful coverage-guided fuzzer.
- **Syzkaller** → Specially for Linux kernel fuzzing.
- **Radamsa** → Simple dumb fuzzer (great for quick tests).
- **Bochs / QEMU** → Used when you don't have source code.
- Python tools → boofuzz (network protocols), PyFuzzer, etc.

Real-Life Success Stories

- **Mozilla Firefox** → Fuzzers found 20 serious bugs (including 5 critical) in 2022–2023 that could have allowed hackers to take over your computer.
- **OpenSSL** → Google's OSS-Fuzz found two critical bugs in 2022 that could crash every server using HTTPS.
- **Linux Kernel** → Syzkaller project finds thousands of kernel bugs every year – many could have been exploited your laptop/phone.
- **Chrome, Edge, PDF readers, media players** → All big companies now run fuzzers 24×7.

Stubs

What is a Stub?

A **stub** is a tiny fake replacement for a real component that is not ready, too slow, too expensive, or not available during testing.

It pretends to be the real thing but only returns **pre-decided, hard-coded (fake) answers**.

Examples everyone understands:

- Fake login module → always says “login successful”
- Fake payment gateway → always returns “payment success” with order ID 12345
- Fake SMS service → just prints “SMS sent to 9876543210” on the screen
- Fake database → always returns the same 5 rows of data

Stubs are used only to **unblock testing** when the real part is missing.

How to Create a Stub – 4 Easy Steps Step 1: Identify the dependency

Look at your class A class is calling something that is problematic or missing.

Example:

- A calls `PaymentGateway.chargeCard()` → but real gateway needs internet/money
- A calls `Database.getUser()` → but database server is not ready → You decide: “I need a stub for this.”

Step 2: Extract the interface

Take only the method(s) that A actually uses and make a small interface.

Example:

Java

```
interface PaymentGateway {  
    String chargeCard(String cardNo, int amount);  
}
```

Now your original code depends only on this interface, not on the real big class.

Step 3: Write the stub class

Create a new tiny class that implements the same interface but returns fake data.

Java

```
class FakePaymentGateway implements PaymentGateway {  
    public String chargeCard(String cardNo, int amount) {  
        return "FAKE_SUCCESS_999"; // always fake success  
    }  
}
```

That's it – the whole stub is usually 5–15 lines.

Step 4: Inject the stub into your code

Give the stub to class A instead of the real one.

Three common ways (choose any one):

1. Through **constructor** (most popular)

Java

```
A a = new A(new FakePaymentGateway());
```

2. Through **setter method**

Java

```
a.setPaymentGateway(new FakePaymentGateway());
```

3. As a **method parameter** (simple for one test)

Java

```
a.processOrder(new FakePaymentGateway());
```